

Artificial Intelligence for Video Games

Voxel-based Terrain Generator

Zong Yuan Alessandro Gong

19167A

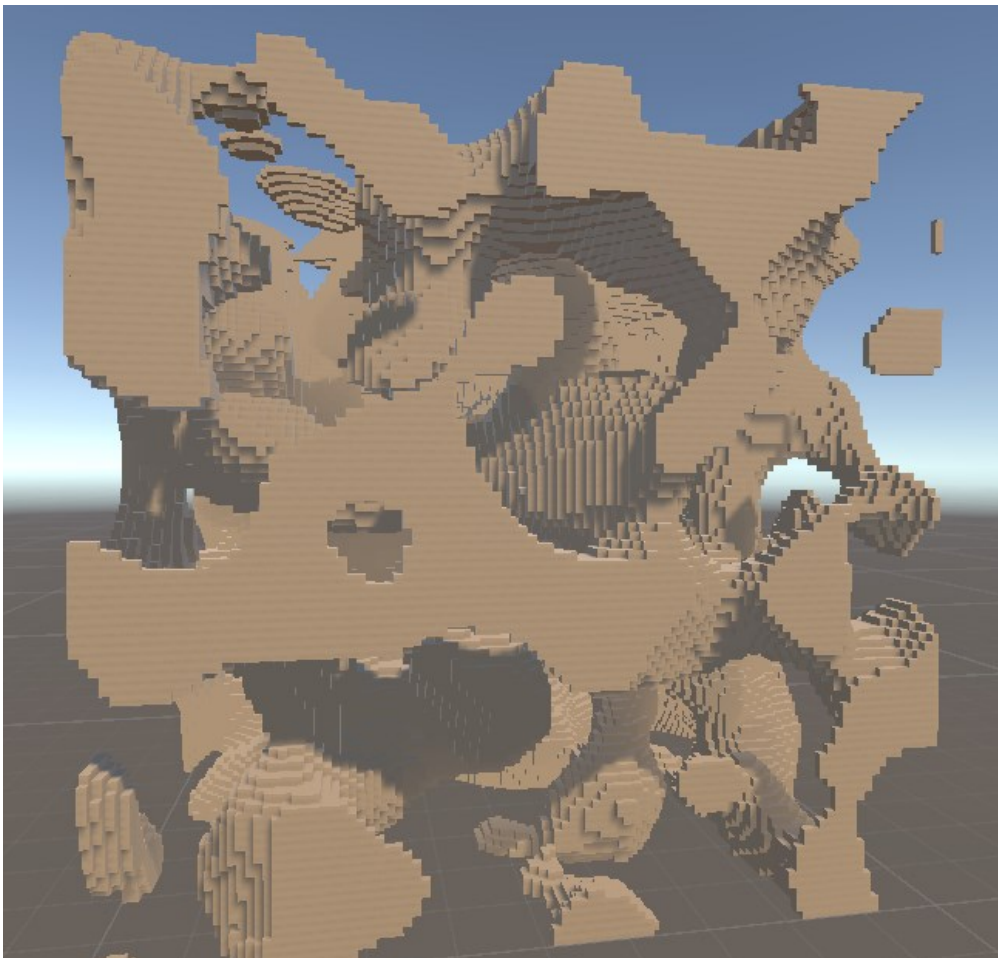
Project description

The project creates a procedural terrain based on voxels.

The terrain is created inside a 3D cube of 200x200x200 meters, with each voxel being 1x1x1 meters.

Each voxel is assigned to rock (color light brown) or air (meaning the voxel isn't rendered). The decision is made based on the comparison between a value of 3D Perlin noise and a threshold.

A “fractal” 3D Perlin noise is used, with a maximum of 5 octaves.



Terrain generation

Terrain is generated calculating the 3D Perlin Noise for each voxel. Then each value is compared with a threshold: if above it, a rock voxel is instantiated, otherwise an air voxel is. Rock voxel is a rendered light brown cube, meanwhile the air voxel is a non-rendered light blue cube. To save resources, there's an option that decides whether the air voxels are instantiated.

The variable *size* controls how big is the terrain. Default is set to 200m.

```
float[,] noise = Amplify(Noise3D(size, size, size, resolution), size, size, size, amplitude);
for (int h = 1; h <= octaves; h++) {
    float[,] octave = Amplify(Noise3D(size, size, size, resolution / (int)System.Math.Pow(2, h)),
        size, size, size, amplitude / (int)System.Math.Pow(2, h));
    noise = Sum(noise, octave, size, size, size);
}

noise = Normalize(noise, size, size, size);

noise = myNoiseFunctions[(int)noiseFunction](noise, size, size, size);

for (int i = 0; i < size; i++) {
    for (int j = 0; j < size; j++) {
        for (int k = 0; k < size; k++) {
            if (noise[i, j, k] > threshold)
                Instantiate(voxelRock, new Vector3(i + 0.5f, j + 0.5f, k + 0.5f), Quaternion.identity);
            else if (createAir)
                Instantiate(voxelAir, new Vector3(i + 0.5f, j + 0.5f, k + 0.5f), Quaternion.identity);
        }
    }
}
```

```
public int size = 200;
```

3D Perlin Noise

The Perlin noise is generated using the function *Noise3D*.

First, the array of gradients is generated, for the specified area at the specified resolution. This is achieved by instantiating a random Vector3. The seed for the Random Generator is set at the start of the program and saved.

```
if (RandomSeed == 0) RandomSeed = (int)System.DateTime.Now.Ticks;  
Random.InitState(RandomSeed);
```

```
int xOut = 1 + resolution * (1 + (int)Mathf.Ceil(x / resolution));  
int yOut = 1 + resolution * (1 + (int)Mathf.Ceil(y / resolution));  
int zOut = 1 + resolution * (1 + (int)Mathf.Ceil(z / resolution));  
  
Vector3[,,] gradients = new Vector3[xOut, yOut, zOut];  
  
for (int i = 0; i < xOut; i += resolution) {  
    for (int j = 0; j < yOut; j += resolution) {  
        for (int k = 0; k < zOut; k += resolution) {  
            gradients[i, j, k] = Random.insideUnitSphere;  
        }  
    }  
}
```

The second half of the function calculated the 3D Perlin Noise matrix for the entire terrain.

```
float[,] noise = new float[x, y, z];

for (int i = 0; i < x; i++) {
    for (int j = 0; j < y; j++) {
        for (int k = 0; k < z; k++) {
            int floorX = resolution * ((int)Mathf.Floor((float)i / resolution));
            int floorY = resolution * ((int)Mathf.Floor((float)j / resolution));
            int floorZ = resolution * ((int)Mathf.Floor((float)k / resolution));
            int ceilX = resolution * ((int)Mathf.Ceil((float)i / resolution));
            int ceilY = resolution * ((int)Mathf.Ceil((float)j / resolution));
            int ceilZ = resolution * ((int)Mathf.Ceil((float)k / resolution));

            float h1 = Vector3.Dot(gradients[floorX, floorY, floorZ], new Vector3(i, j, k) - new Vector3(floorX, floorY, floorZ));
            float h2 = Vector3.Dot(gradients[floorX, floorY, ceilZ], new Vector3(i, j, k) - new Vector3(floorX, floorY, ceilZ));
            float h3 = Vector3.Dot(gradients[floorX, ceilY, floorZ], new Vector3(i, j, k) - new Vector3(floorX, ceilY, floorZ));
            float h4 = Vector3.Dot(gradients[floorX, ceilY, ceilZ], new Vector3(i, j, k) - new Vector3(floorX, ceilY, ceilZ));
            float h5 = Vector3.Dot(gradients[ceilX, floorY, floorZ], new Vector3(i, j, k) - new Vector3(ceilX, floorY, floorZ));
            float h6 = Vector3.Dot(gradients[ceilX, floorY, ceilZ], new Vector3(i, j, k) - new Vector3(ceilX, floorY, ceilZ));
            float h7 = Vector3.Dot(gradients[ceilX, ceilY, floorZ], new Vector3(i, j, k) - new Vector3(ceilX, ceilY, floorZ));
            float h8 = Vector3.Dot(gradients[ceilX, ceilY, ceilZ], new Vector3(i, j, k) - new Vector3(ceilX, ceilY, ceilZ));

            float u = ((float)i - floorX) / resolution;
            float v = ((float)j - floorY) / resolution;
            float w = ((float)k - floorZ) / resolution;

            float l1 = Mathf.Lerp(h1, h5, slope(u));
            float l2 = Mathf.Lerp(h2, h6, slope(u));
            float l3 = Mathf.Lerp(h3, h7, slope(u));
            float l4 = Mathf.Lerp(h4, h8, slope(u));

            float m1 = Mathf.Lerp(l1, l3, slope(v));
            float m2 = Mathf.Lerp(l2, l4, slope(v));

            noise[i, j, k] = Mathf.Lerp(m1, m2, slope(w));
        }
    }
}
```

For each point in the terrain, the Perlin noise is calculated using the 8 gradients that surround it, calculating their contribution and interpolating them to find the result.

The octaves are calculated by calling the Noise3D function again, doubling the frequency of the subsampling and halving the resulting noise for each octave.

The result is then normalized to the range [0, 1].

```
float[,] noise = Amplify(Noise3D(size, size, size, resolution), size, size, size, amplitude);
for (int h = 1; h <= octaves; h++) {
    float[,] octave = Amplify(Noise3D(size, size, size, resolution / (int)System.Math.Pow(2, h)),
        size, size, size, amplitude / (int)System.Math.Pow(2, h));
    noise = Sum(noise, octave, size, size, size);
}

noise = Normalize(noise, size, size, size);
```

```
private float [,,] Normalize(float[,,] m, int x, int y, int z) {  
    float max, min;  
    max = float.MinValue;  
    min = float.MaxValue;  
  
    for (int i = 0; i < x; i++) {  
        for (int j = 0; j < y; j++) {  
            for (int k = 0; k < z; k++) {  
                if (m[i, j, k] < min) min = m[i, j, k];  
                if (m[i, j, k] > max) max = m[i, j, k];  
            }  
        }  
    }  
    for (int i = 0; i < x; i++) {  
        for (int j = 0; j < y; j++) {  
            for (int k = 0; k < z; k++) {  
                m[i, j, k] = (m[i, j, k] - min) / (max - min);  
            }  
        }  
    }  
    return m;  
}
```

Noise functions

Some noise functions have been implemented, to achieve different results in the terrain generation.

```
public enum NoiseFunctions { Identity, Inverse, Square, Root, HeightFalloff };  
1 reference  
public NoiseFunction[] myNoiseFunctions = { Identity, Inverse, Square, Root, HeightFalloff };  
1 reference  
public NoiseFunctions noiseFunction = NoiseFunctions.Identity;
```

The functions are applied on the normalized noise.

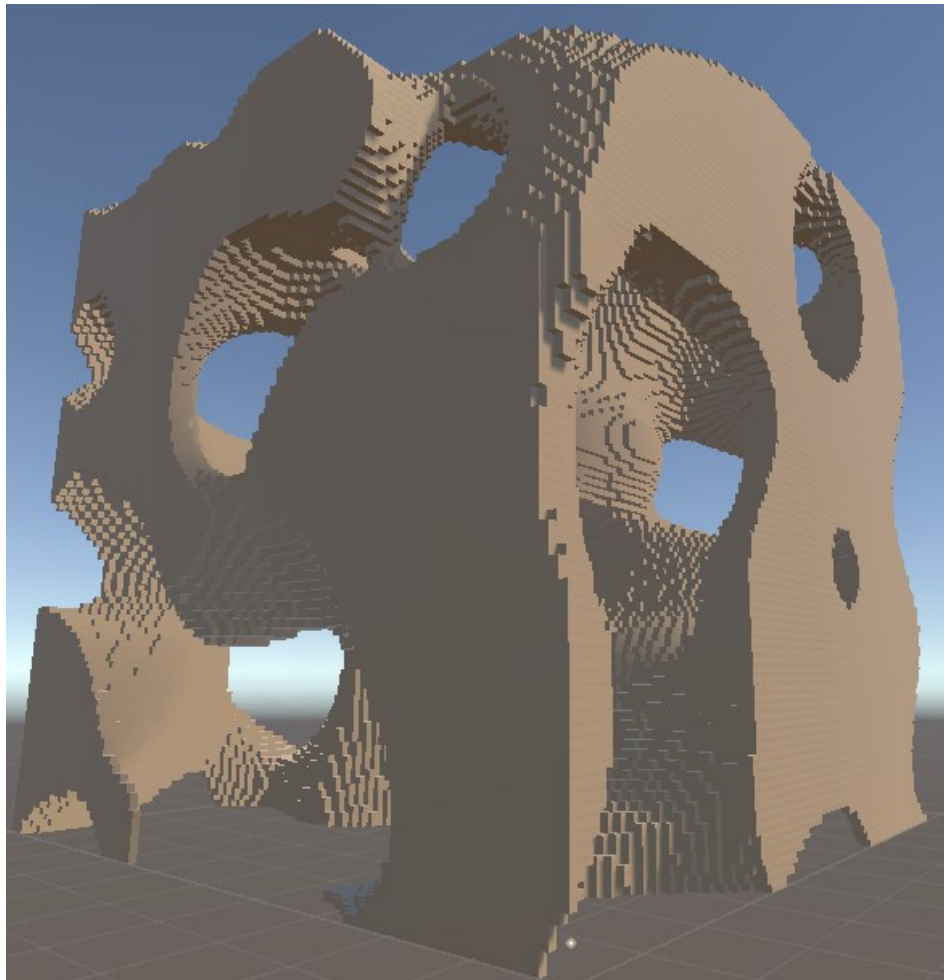
```
noise = Normalize(noise, size, size, size);  
  
noise = myNoiseFunctions[(int)noiseFunction](noise, size, size, size);
```

The following examples are obtained using the following configuration.

Size	100
Octaves	<input type="range"/> 0
Threshold	<input type="range"/> 0.5
Resolution	32
Amplitude	1
Create Air	☐
Random Seed	886112160

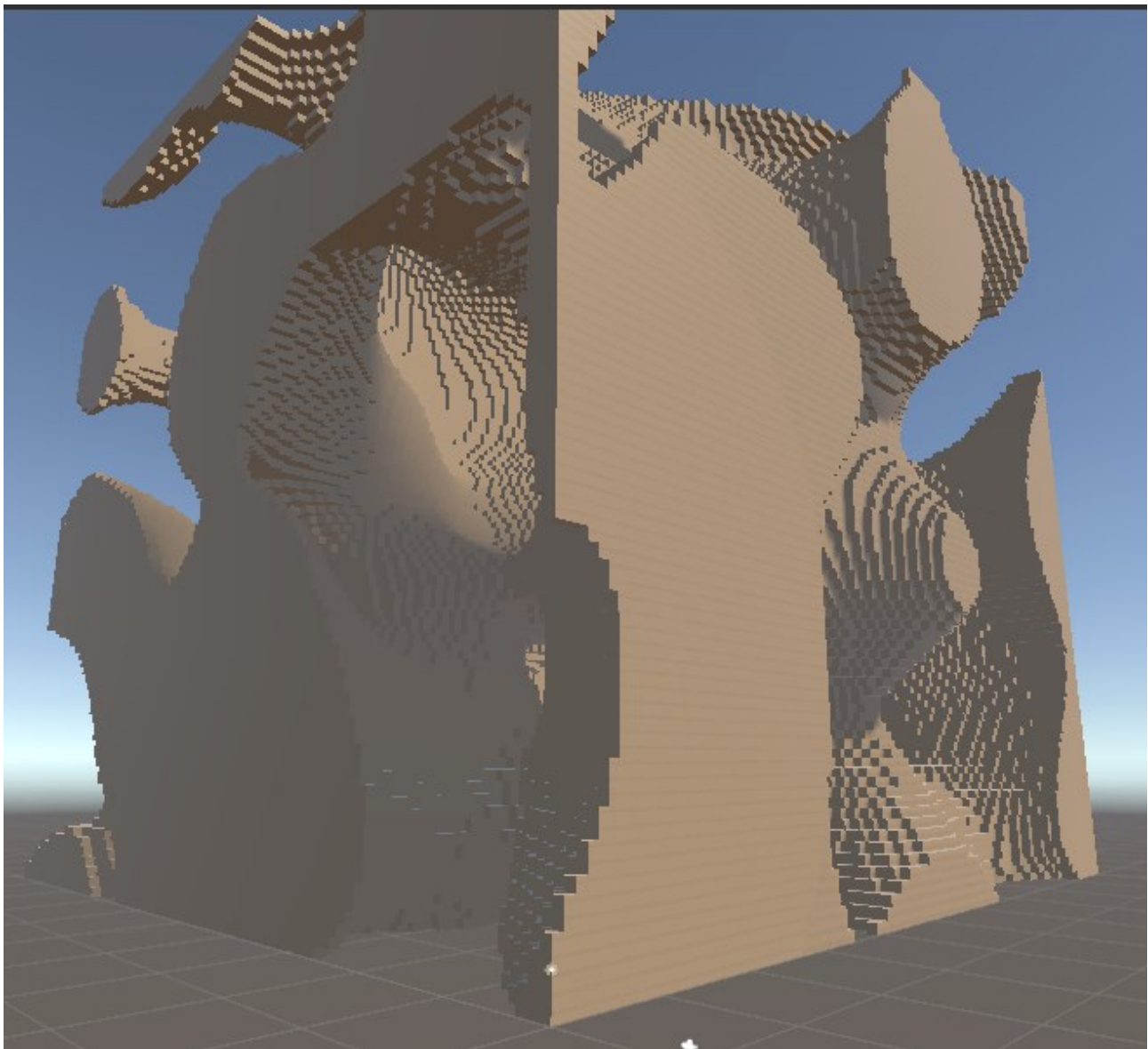
The Identity function simply returns the obtained noise value.

```
private static float[, ,] Identity(float[, ,] noise, int x, int y, int z) {  
    return noise;  
}
```



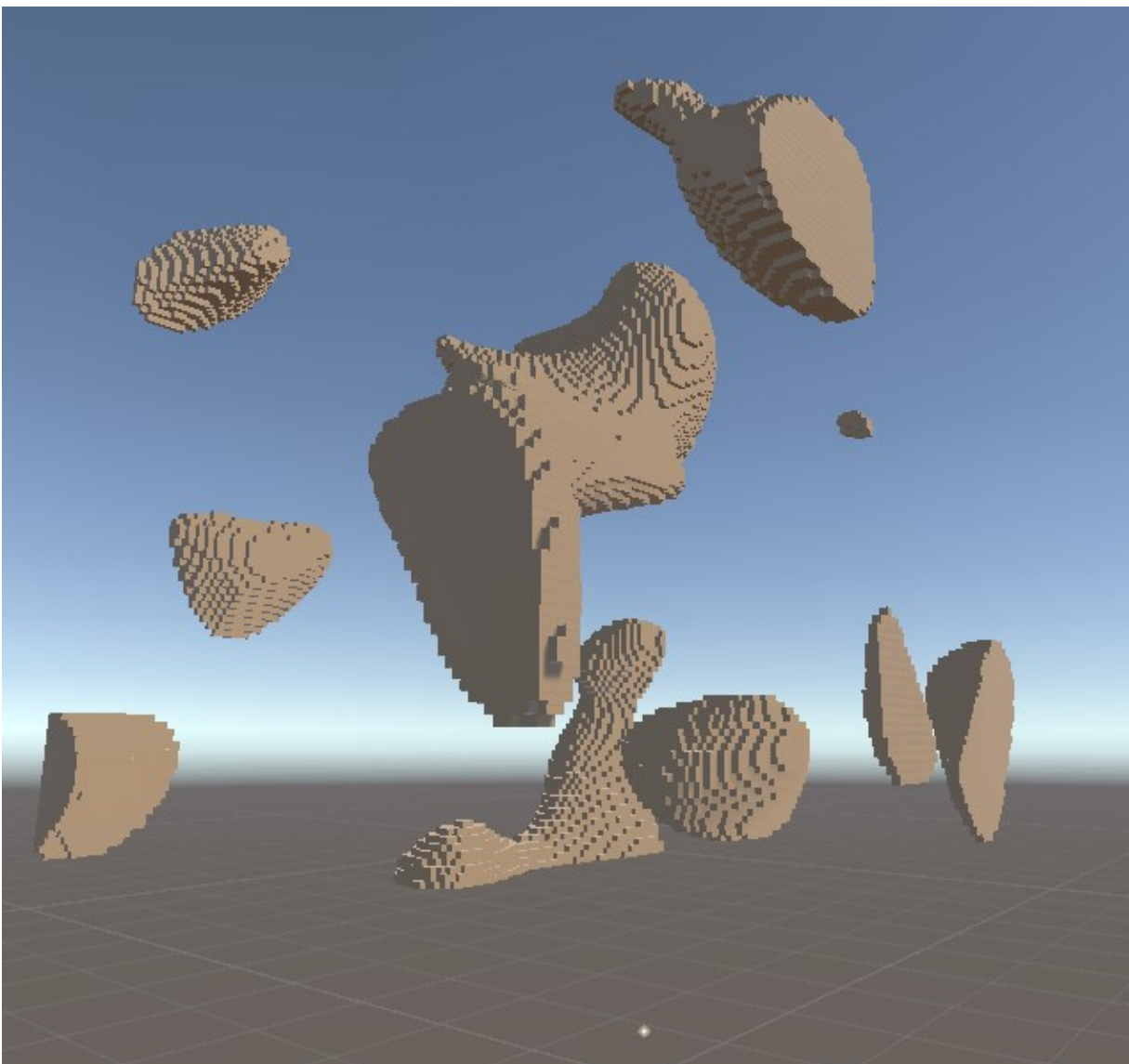
The Inverse function inverts the result of the noise, flipping the terrain generation.

```
private static float[,,] Inverse(float[,,] noise, int x, int y, int z) {  
    for (int i = 0; i < x; i++) {  
        for (int j = 0; j < y; j++) {  
            for (int k = 0; k < z; k++) {  
                noise[i, j, k] = 1f - noise[i, j, k];  
            }  
        }  
    }  
    return noise;  
}
```



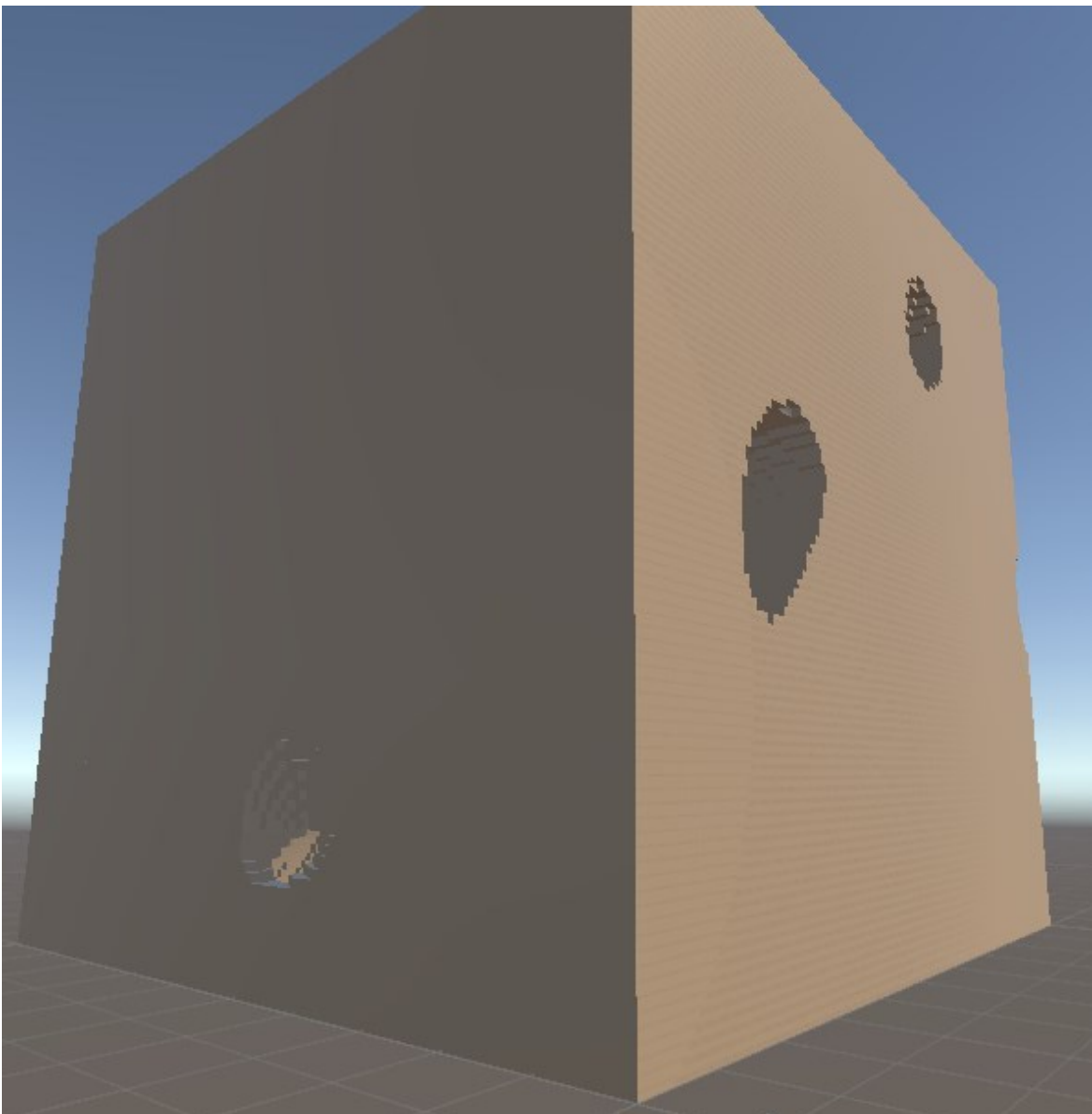
The Square function applies the square power to the noise value. This increases the empty spaces, making it more suitable for sky islands terrains.

```
private static float[, ,] Square(float[, ,] noise, int x, int y, int z) {  
    for (int i = 0; i < x; i++) {  
        for (int j = 0; j < y; j++) {  
            for (int k = 0; k < z; k++) {  
                noise[i, j, k] = Mathf.Pow(noise[i, j, k], 2);  
            }  
        }  
    }  
    return noise;  
}
```



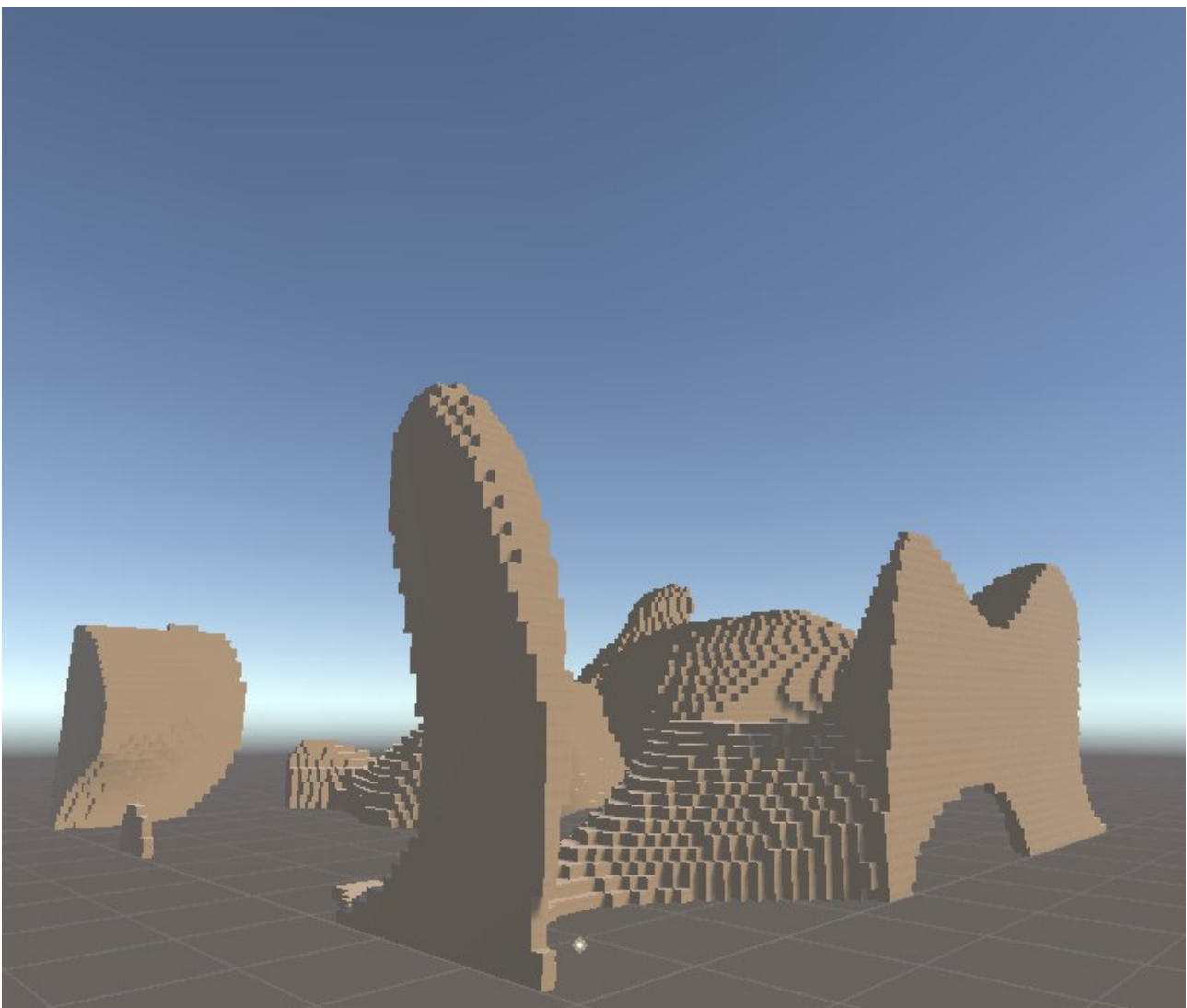
The Root function does the opposite of the Square function, increasing instead the number of rock voxels. This function is more useful to generate underground terrain, focusing on the creation of caves.

```
private static float[, ,] Root(float[, ,] noise, int x, int y, int z) {  
    for (int i = 0; i < x; i++) {  
        for (int j = 0; j < y; j++) {  
            for (int k = 0; k < z; k++) {  
                noise[i, j, k] = Mathf.Pow(noise[i, j, k], 0.5f);  
            }  
        }  
    }  
    return noise;  
}
```

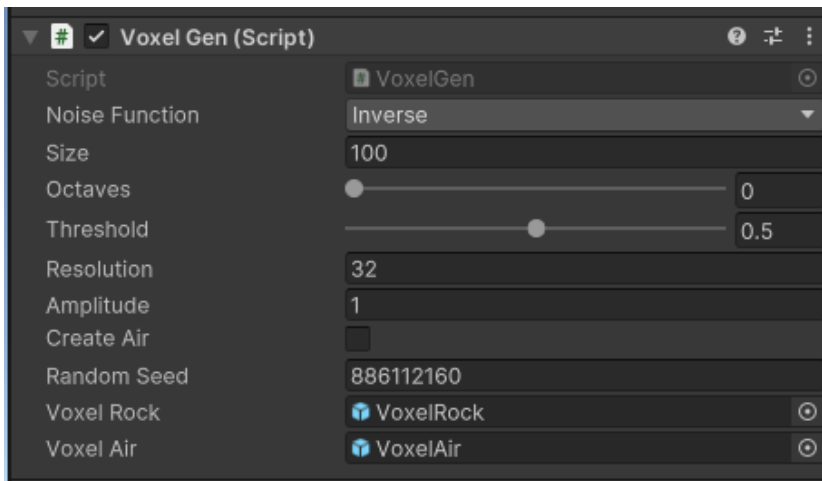


The HeightFalloff function allows for the generation of a terrain with clear separation between the sky and the earth, focusing on generating hills and mountains.

```
private static float[, ,] HeightFalloff(float[, ,] noise, int x, int y, int z) {  
    for (int i = 0; i < x; i++) {  
        for (int j = 0; j < y; j++) {  
            for (int k = 0; k < z; k++) {  
                float falloff = 1f - ((float)j / y);  
                noise[i, j, k] *= falloff;  
            }  
        }  
    }  
    return noise;  
}
```



Parameters



Various parameters are used to control the generation.

Noise function selects the used function applies on the noise values.

Size controls the size of the terrain.

Octaves is an integer controlling the number of octaves used (from 0 to 5).

Threshold is a $[0, 1]$ number and the minimum value to be considered a rock voxel.

Resolution and Amplitude are used to finetune the values for the noise.

Create Air is the Boolean value to decide whether to instantiate air voxels.

Problems and possible improvements

Because of the very big size of the terrain, there are a lot of performance issues to be dealt with. My PC couldn't handle all the voxels, so I was forced to downsize to at most 100m cube, but I left a variable to modify the size.

One possible solution for the performance issues derived from the generation of a high number of voxels could be the use chunks, like in Minecraft. By dividing the terrain in chunks, only some of them would be loaded each time, easing the stress on the device.

Another possible optimization could be combining the visible voxels' faces into meshes, instead of instantiating each voxel. This way, only the combined mesh would be rendered, hiding all the non-visible voxels' faces.